

SecureDove

By: Joshua Samuels, Zachary Mullin, Micha Mettler, Jodie Butterworth, Nicole Fuller, and Ryan Brombaugh

Contributions

Joshua Samuels - Network communication, some frontend, DoS attack and scripting

Micha Mettler - Frontend

Zachary Mullin - Encryption and hashing code, some networking code, Man-In-The-Middle scripting

Nicole Fuller - Testing private key leakage

Ryan Brombaugh - ID transferring code, Wireshark Man-In-The-Middle hijack

Jodie Butterworth - Nothing

Our Project

Our project was a secure and encrypted messaging system.

- Users can send messages securely over the internet, with a guarantee that the message they are sending will be Confidential, will not be altered (Integrity), and will be Available on demand. CIA.
- It is designed for 1-on-1 communications between two users.

Building the First Prototype

Starting Ideas:

- Started out with a peer-to-peer system where one user would have to port-forward their device on port 50001, and then another user could connect.
- This was later changed so that users could connect to a central server and input a shared password that would be decided offline to connect to one another.
- All messages would be end-to-end encrypted by creating a shared secret and using the ChaCha20-Poly1305 encryption algorithm, which ensures confidentiality and integrity by encrypting messages and digitally signing them to prevent forgery. (This is the same algorithm used in TLS)

Found Vulnerabilities

Transmitting Passwords in Plaintext.

- While in concept our idea of creating rooms you could only access with a password that was decided offline was secure, we did not encrypt messages between the client and the server. This allowed anybody with a network protocol analyzer to see said password by capturing the network traffic and connect with the password as if they are the intended message recipient.

DoS vulnerability

- We did not add any protections against DoS attacks, and were easily able to deny use of the server by overwhelming it with purposefully incomplete requests.

Fixing the Vulnerabilities

Changes made to accommodate improved security:

- There is now an encrypted handshake between the client and server programs for sending passwords.
- Users now use an authentication system with a username and password as well as a user ID for connecting, rather than connecting with a shared password.
- Passwords are hashed on the server to ensure that if the server was compromised, the passwords are not stolen.
- A LAN solution is now available using Hamachi to connect users without the need of a central server that can be DDOSed and rendered unavailable.

Demonstration Video

<https://youtu.be/VJehZ9fV5IE>

Reflection

Our system changed a lot as a result of decisions we made as a team as for what we thought would fulfill the three security requirements of confidentiality, integrity and availability.

We had to make some concessions in regard to how we designed this, including needing to use Hamachi for the peer-to-peer system, since there was not a good connectionless protocol that allowed for message transfer.

If we did this again, we would probably look for novel ways to design the system in a way that would allow for more than one central server or allow peer-to-peer connections without needing to port-forward a server or use Hamachi.